一、签到

上传含校徽图片后直接获得 flag（这张图片应该算有吧 hhh）

二、babyRSA

1. 分析源代码，发现是将长度为 30 的 flag 分成等长的两部分，然后组合出 c1 和 c2，解出 c1 和 c2 是可以反推出 F1 和 F2 的

```python
assert(len(flag) == 38)

half = len(flag) / 2

flag1 = flag[:half]
flag2 = flag[half:]

secret_num = getPrime(1024) * bytes_to_long(secret)

p = sympy.nextprime(secret_num)
q = sympy.nextprime(p)

N = p * q

e = 0x10001

F1 = bytes_to_long(flag1)
F2 = bytes_to_long(flag2)

c1 = F1 + F2
c2 = pow(F1, 3) + pow(F2, 3)
assert(c2 < N)

m1 = pow(c1, e, N)
m2 = pow(c2, e, N)

output = open('secret', 'w')
output.write('N=' + str(N) + '\n')
output.write('m1=' + str(m1) + '\n')
output.write('m2=' + str(m2) + '\n')
output.close()
```

2. 然后观察到这个 RSA 加密系统，它是获取的两个相邻的大素数，这个东西显然是有漏洞的，这篇文章详细说明了：[RSA-p 和 q 挨得很近(费马分解)_rsa 费马分解-CSDN 博客]。
3. 所以思路就是先用费马分解找到 p 和 q，然后推出 phi、d，解密出 c1、c2，最后变换出 F1、F2，脚本如图所示。

```python
from math import isqrt
from Crypto.Util.number import long_to_bytes

N = 63658514959457474690903016018269086622290925646485
m1 = 43701891055537775475871309067482843455262360941 8
m2 = 42377515688734577148321407317456356622597189937 8
e = 0x10001

def fermat_factor(N):  1个用法
    A = isqrt(N)
    if A * A < N:
        A += 1
    while True:
        t = A*A - N
        b = isqrt(t)
        if b*b == t:
            p = A - b
            q = A + b
            return p, q
        A += 1

p, q = fermat_factor(N)

phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)

c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
P = (c1**3 - c2) // (3 * c1)
D = c1*c1 - 4*P
sqrtD = isqrt(D)
F1 = (c1 + sqrtD) // 2
F2 = (c1 - sqrtD) // 2

b1 = long_to_bytes(F1)
b2 = long_to_bytes(F2)
print(b1,b2)
```

4. 运行得到结果，组合一下就可以得到 flag

```
b'_asdasfasqwe_dmqwo}' b'BITs2CTF{isjna_cask'
```

三、easyAES

1. 分析源代码，就是简单的 aes 加密，但是打印了 key 和 iv 异或的结果，由于一个 32 字节，一个 16 字节，所以有一半是没有被异或的，直接可以得到一半的 key，同时又因为 key 是由一个 2 字节的数重复 16 次得到的，所以 key 和 iv 都可以直接得到了，然后就解密就行了。

```python
from Cryptodome.Cipher import AES
import os
import gmpy2
from flag import FLAG
from Cryptodome.Util.number import *

def main():  1个用法
    key = os.urandom(2) * 16
    iv = os.urandom(16)
    print(bytes_to_long(key) ^ bytes_to_long(iv))
    aes = AES.new(key, AES.MODE_CBC, iv)
    enc_flag = aes.encrypt(FLAG)
    print(enc_flag)

if __name__ == "__main__":
    main()



#output:
#54755426038978086460310747316994556563126341157253266647920514387148480709140
#b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'
```

2. 脚本如图所示，思路就和前面讲得一样

```python
from Cryptodome.Util.number import long_to_bytes
from Cryptodome.Cipher import AES

T = 54755426038978086460310747316994556563126341157253266647920514387148480709140
ciphertext = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

top = T >> 128
bottom = T & ((1 << 128) - 1)
top_bytes = long_to_bytes(top, blocksize: 16)
a, b = top_bytes[0], top_bytes[1]
K_half = bytes([a, b]) * 8
full_key = bytes([a, b]) * 16
bottom_bytes = long_to_bytes(bottom, blocksize: 16)
IV = bytes(x ^ y for x, y in zip(K_half, bottom_bytes))

aes = AES.new(full_key, AES.MODE_CBC, IV)
flag = aes.decrypt(ciphertext)
print(flag)
```

3. 得到 flag

```
b'BITs2CTF{sjcihd_enf_huidjx_hsid}'
```

四、remaining thing

1. 没琢磨出来，整个题直接丢给 gpt，分析得非常好啊，确实是考的中国剩余定理

C3=41663026452800366344084116174180504236555705080505184877204086431216138707209002745841467383559886160630317645159881729713488002376046452536102884351087721916511569274797438511760002651174638774663418515491471728733423349429301126162379031930269212 4

这是典型的 e=3 广播攻击（Hastad's Broadcast Attack）场景：同一个明文 m 用相同的 e=3 在三个互不相同的模数 N1, N2, N3 下加密，得到 C1=m^3 mod N1, C2=m^3 mod N2, C3=m^3 mod N3。用中国剩余定理（"中国剩余定理"的"剩""余"正是题目与诗句的提示）将三个同余合并可得到 m^3 的真值，再对其取整数立方根即可恢复 m。

下面给你一份纯标准库的 Python 脚本，自动尝试两条路：

- 首选：Hastad 广播攻击（CRT 合并后取整数立方根）
- 兜底：如果发现模数之间有公因子（gcd>1），则对可分解的模数直接用 d=e^{-1} mod phi 解密

运行后会打印出可能的明文（bytes 和尝试解码为 utf-8/latin-1 的字符串）。

solve_remaining_thing.py

```python
#!/usr/bin/env python3

from math import gcd

# ====== 输入（题目给定）======
e = 3
```

使用方法

- 将上面脚本保存为 solve_remaining_thing.py
- 直接运行：python3 solve_remaining_thing.py
- 查看输出中解出的明文（一般会直接包含 flag）。

思路小结

- 题面与诗句提示"剩""余"，指向"中国剩余定理"。
- e=3 且同明文多模数加密 → 用 CRT 复原 m^3 → 取整数立方根得到 m（Hastad 广播攻击）。
- 若模数之间有公因子，则走 gcd 分解 → 计算 d → 常规 RSA 解密。

2. 既然脚本都帮我写出来了，直接运行，得到 flag（ai 太好用了你知道吗 ：D）

```python
#!/usr/bin/env python3

from math import gcd

# ====== 输入（题目给定）======
e = 3

N1 = 19831146070473780450608271965513333016477420167810804693209535211
C1 = 13192445407076444144417541514427188094854353253584529650871742429

N2 = 66111404265300574613935728983655950688111510106384565086238514901
C2 = 41998123843875621116729989501531316956049116280732482300490270393

N3 = 14126749535297606509514418674348551918358936748316341471639452521
C3 = 41663026452800366344084116174180504236555705080505184877204086431

Ns = [N1, N2, N3]
Cs = [C1, C2, C3]


def invmod(a, m):  2 用法
    # Python 3.8+ supports pow(a, -1, m) for modular inverse
    return pow(a, -1, m)


def crt(remainders, moduli):  1 个用法
    # x ≡ a_i (mod n_i), with pairwise coprime n_i
    N = 1
    for n in moduli:
        N *= n
    x = 0
    for a, n in zip(remainders, moduli):
        Ni = N // n
        inv = invmod(Ni, n)
        x = (x + a * Ni * inv) % N
    return x, N


def iroot3(n: int):  1 个用法
    # Integer cube root via binary search: returns floor(cuberoot(n))
    lo, hi = 0, 1
```

```python
def iroot3(n: int):  1 个用法
    # Integer cube root via binary search: returns floor(cuberoot(n))
    lo, hi = 0, 1
    # Exponentially expand hi
    while hi * hi * hi <= n:
        hi <<= 1
    # Binary search
    while lo + 1 < hi:
        mid = (lo + hi) // 2
        m3 = mid * mid * mid
        if m3 <= n:
            lo = mid
        else:
            hi = mid
    return lo


def to_bytes(n: int) -> bytes:  2 用法
    if n == 0:
        return b"\x00"
    length = (n.bit_length() + 7) // 8
    return n.to_bytes(length, byteorder: 'big')


def try_decode(b: bytes):  2 用法
    outs = []
    for enc in ("utf-8", "latin-1"):
        try:
            outs.append((enc, b.decode(enc)))
        except UnicodeDecodeError:
            pass
    return outs


def broadcast_attack_hastad(Cs, Ns, e=3):  1 个用法
    # Assumes same plaintext under different pairwise-coprime moduli
    m3, bigN = crt(Cs, Ns)
    m = iroot3(m3)
    if m * m * m != m3:
        return None, None, None  # not a perfect cube
    return m, m3, bigN
```

```python
def common_factor_attack(Ns, Cs, e=3):  1 个用法
    # If any gcd(Ni, Nj) > 1, factor and decrypt those instances
    results = []
    k = len(Ns)
    for i in range(k):
        for j in range(i + 1, k):
            g = gcd(Ns[i], Ns[j])
            if g != 1 and g != Ns[i] and g != Ns[j]:
                # Factor Ni, Nj
                p = g
                for idx in (i, j):
                    N = Ns[idx]
                    q = N // p
                    phi = (p - 1) * (q - 1)
                    try:
                        d = invmod(e, phi)
                    except ValueError:
                        continue
                    m = pow(Cs[idx], d, N)
                    results.append((idx + 1, N, m))
    return results


def main():  1 个用法
    print("[-] Checking pairwise gcds...")
    for i in range(len(Ns)):
        for j in range(i + 1, len(Ns)):
            print(f"gcd(N{i+1}, N{j+1}) = {gcd(Ns[i], Ns[j])}")

    print("\n[-] Trying Hastad broadcast attack (CRT + integer cube root)...")
    try:
        m, m3, bigN = broadcast_attack_hastad(Cs, Ns, e)
    except ValueError as ex:
        print(f"[!] CRT failed with error: {ex}")
        m = None

    if m is not None:
        mb = to_bytes(m)
        print("[+] Hastad attack succeeded.")
        print(f"m (int) = {m}")
        print(f"m (hex) = {mb.hex()}")
```

```python
    if m is not None:
        mb = to_bytes(m)
        print("[+] Hastad attack succeeded.")
        print(f"m (int) = {m}")
        print(f"m (hex) = {mb.hex()}")
        print(f"m (len bytes) = {len(mb)}")
        decs = try_decode(mb)
        for enc, txt in decs:
            print(f"[+] Decoded as {enc}: {txt}")
    else:
        print("[!] Hastad attack did not yield a perfect cube. Trying common-factor attack...")

        results = common_factor_attack(Ns, Cs, e)
        if not results:
            print("[!] No common factors found or decryption failed.")
        else:
            for idx, N, m in results:
                mb = to_bytes(m)
                print(f"[+] Decrypted for N{idx}:")
                print(f"    m (int) = {m}")
                print(f"    m (hex) = {mb.hex()}")
                print(f"    m (len bytes) = {len(mb)}")
                decs = try_decode(mb)
                for enc, txt in decs:
                    print(f"    Decoded as {enc}: {txt}")


if __name__ == "__main__":
    main()
```

3. 得到 flag

```
[+] Decoded as utf-8: BITs2CTF{^_^cRVni_aXinSsd_fJnebvooo^_^}
```

五、签顺道

1. 64 位，直接拖进 ida,

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  int v4; // [rsp+Ch] [rbp-114h]
  char s[264]; // [rsp+10h] [rbp-110h] BYREF
  unsigned __int64 v6; // [rsp+118h] [rbp-8h]

  v6 = __readfsqword(0x28u);
  memset(s, 0, 0x100uLL);
  puts(asc_21C0);
  puts(asc_21DF);
  putchar(62);
  putchar(32);
  v4 = read(0, s, 0x100uLL);
  if ( s[v4 - 1] == 10 )
    s[v4 - 1] = 0;
  if ( !strcmp(s, "f8fqgfm") )
  {
    puts("鍋┣紝瀆逛簡");
    system("/bin/sh");
  }
  else
  {
    puts(asc_2218);
  }
  return 0;
}
```

2. 很简单啊，直接发 f8fqgfm 就可以了

```python
from pwn import *

HOST = "127.0.0.1"
PORT = 4716

context.encoding = 'utf-8'
context.log_level = 'debug'

io = remote(HOST, PORT)
io.recvuntil(b'> ')
io.sendline(b'f8fqgfm')

io.interactive()
```

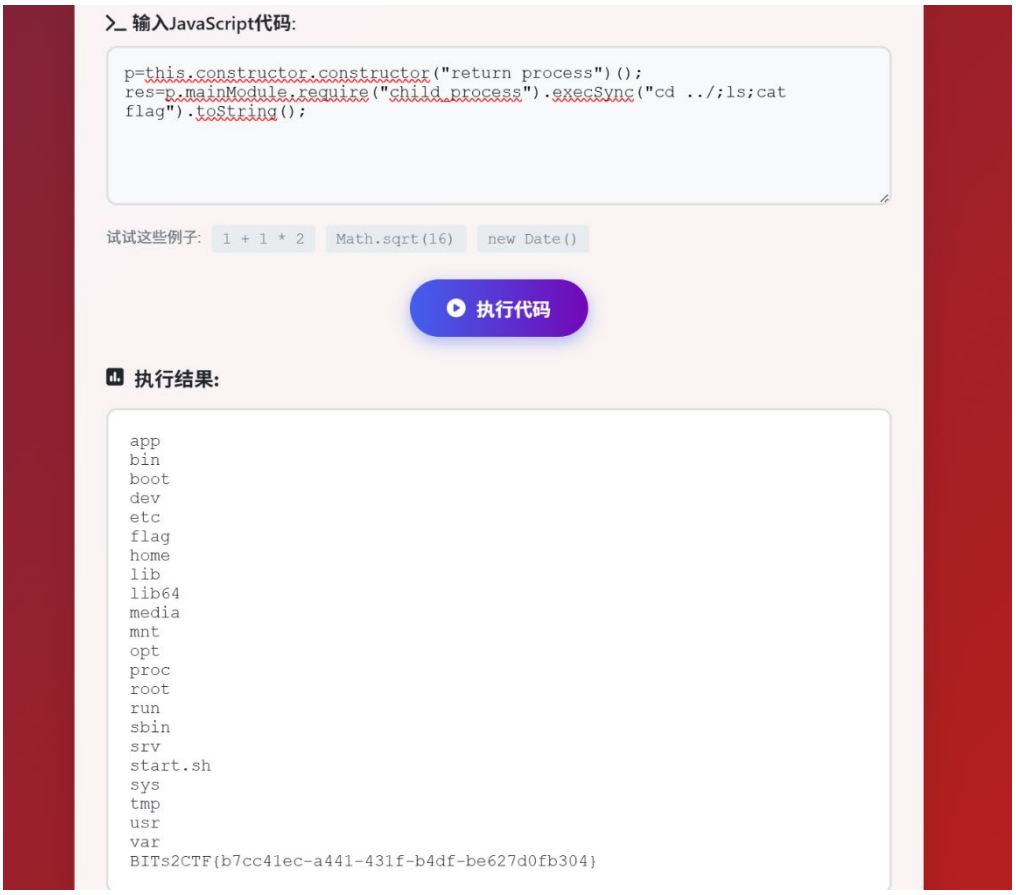
3. 不停 cd ../;ls，最后找到 flag

```
[DEBUG] Sent 0x1 bytes:
    b'a'
[DEBUG] Sent 0x1 bytes:
    b'g'
[DEBUG] Sent 0x1 bytes:
    b'\n'
[DEBUG] Received 0x54 bytes:
    b'BITs2CTF{thaNKs_for_6it5_Fla6_HoUhoUHOu_7ef3136b6c04_90oD_Br0Thers_we_4REHouH0uH0u}\n'
BITs2CTF{thaNKs_for_6it5_Fla6_HoUhoUHOu_7ef3136b6c04_90oD_Br0Thers_we_4REHouH0uH0u}
```

六、1zvm

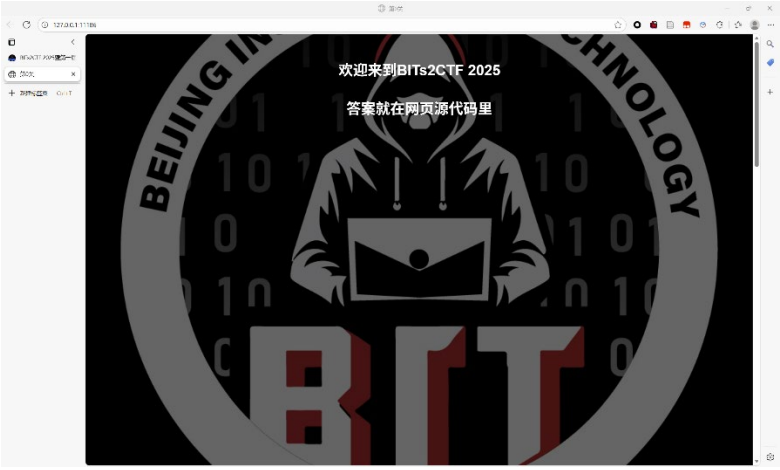
1. 没咋学过这个，但是看题目应该是 js 沙箱的什么漏洞，然后搜索框里找到 js 沙箱逃逸这个关键词，搜到一篇文章：[CTF 题型 nodejs(2) Js 沙盒逃逸原理&典型例题 | J1rrY's Blog]。
2. 直接用它给出的最简单的一个 payload，直接给到 root 了

3. 那直接开始向上遍历，找到 flag

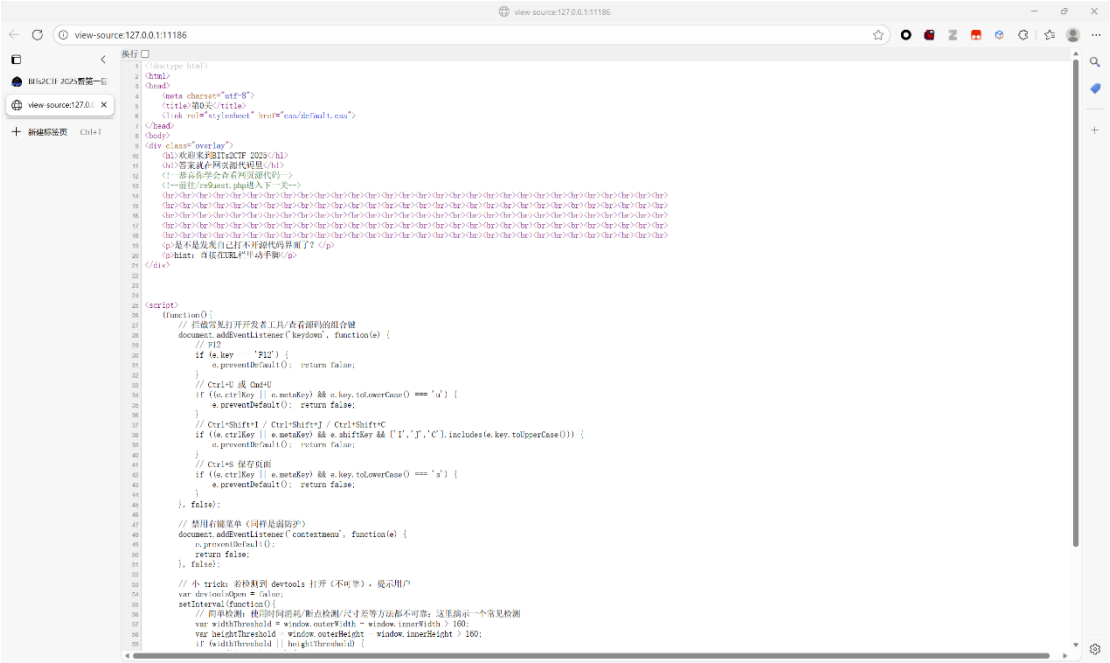


七、http

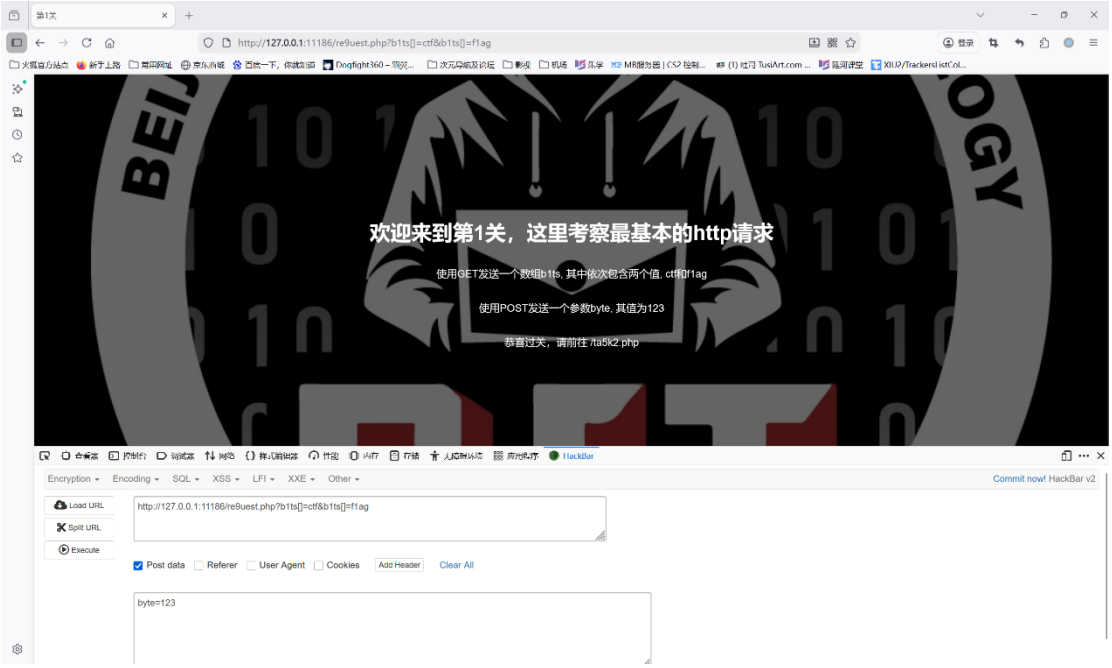
1. 经典的不能 f12



2. 直接“view-source:”，给到下一关“/re9uest.php”



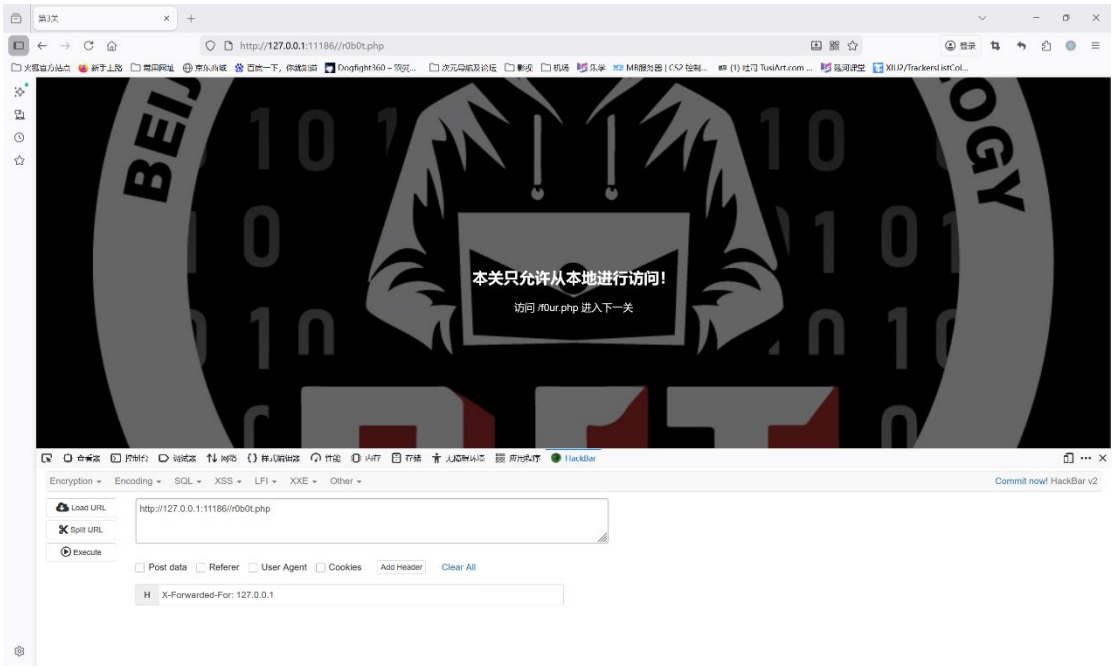
3. 是考 get 和 post 传参（换了下浏览器，因为 edge 没装 hackbar），下一关“/ta5k2.php”



4. 即答：robots.txt，下一关“r0b0t.php”

```
User-agent: *
Disallow: /r0b0t.php


User-agent: Yandex
Disallow: *
```

5. 改 Header 就行了，下一关：“/f0ur.php”



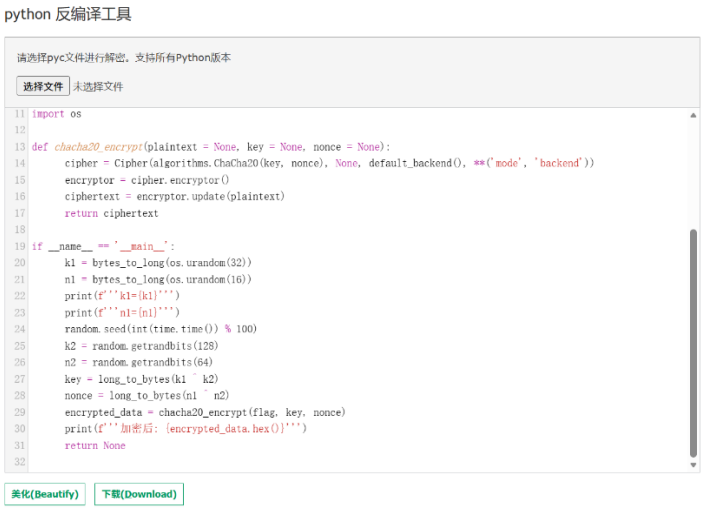
6. 点开 cookie 一看，果然有一个值叫 admin

admin	0	127.0.0.1
PHPSESSID	a501e8c02be469d46c024530f816a3bd	127.0.0.1

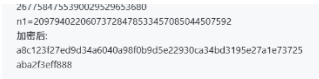
7. 改了过后拿到奖励，是 base64，解码拿到 flag

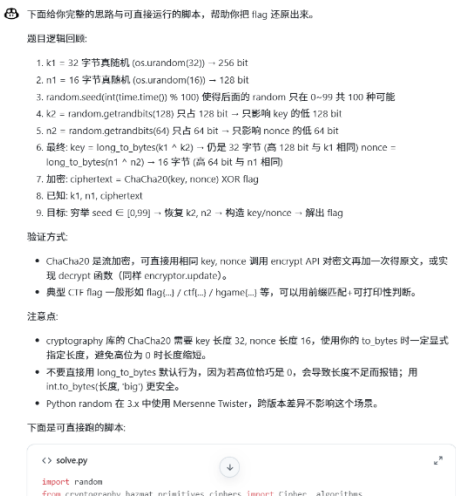
八、ChaCha20

1. 直接 pyc 反编译，得到 python 源码



2. 这个加密没听过，直接 gpt 一键扫荡（）





3. 获得脚本

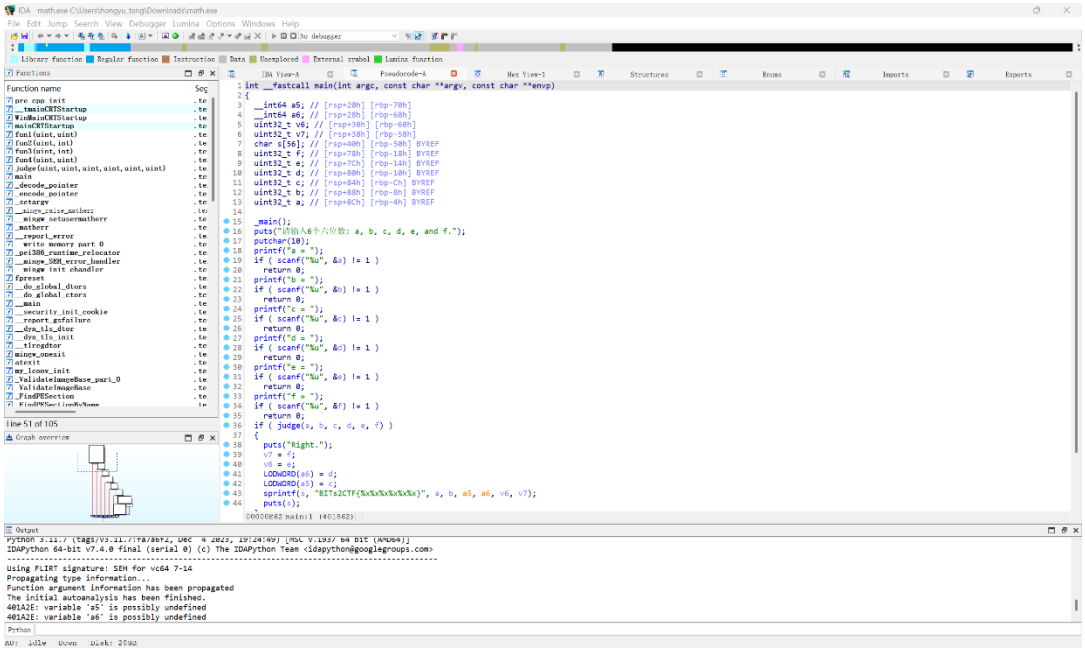
```python
import random
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend

# 已知数据
k1 = 89156737880809474145449532029493055444849328922741582677584755390029529653680
n1 = 209794022060737284785334570850445075 92
cipher_hex = "a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888"

cipher_bytes = bytes.fromhex(cipher_hex)

def chacha20_encrypt(data: bytes, key: bytes, nonce: bytes) -> bytes:  1 个用法
    cipher = Cipher(algorithms.ChaCha20(key, nonce), mode: None, default_backend())
    encryptor = cipher.encryptor()
    return encryptor.update(data)

def try_seed(seed: int):  1 个用法
    random.seed(seed)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)

    # 构造 key 与 nonce
    key_int = k1 ^ k2
    nonce_int = n1 ^ n2

    key_bytes = key_int.to_bytes( length: 32,  byteorder: 'big')
    nonce_bytes = nonce_int.to_bytes( length: 16,  byteorder: 'big')

    # 解密：再次加密密文即可还原原文（流密码）
    plaintext = chacha20_encrypt(cipher_bytes, key_bytes, nonce_bytes)

    return plaintext

def looks_like_flag(pt: bytes):  1 个用法
    # 可根据实际 CTF 修改
    text = None
    try:
        text = pt.decode( encoding: 'utf-8', errors='ignore')
    except:
        return False
    if not all(32 <= c <= 126 for c in pt):  # 全部可打印
        return False
    prefixes = ["flag{", "ctf{", "hgame{", "ACS{", "AIS3{", "Cyber{", "qwq{", "shell{", "sec{", "INCTF{"]
    if any(text.startswith(p) for p in prefixes):
        return True
    # 或者长度与密文相符且含花括号模式
    if "{" in text and "}" in text and len(pt) <= 100:
        return True
    return False

candidates = []
for s in range(100):
    pt = try_seed(s)
    if looks_like_flag(pt):
        candidates.append((s, pt))

if not candidates:
    print("未找到明显 flag. 建议打印全部结果人工查看或放宽规则。")
else:
    for seed, pt in candidates:
        print(f"[+] seed={seed} => flag? {pt.decode( encoding: 'utf-8', errors='ignore')}")
```

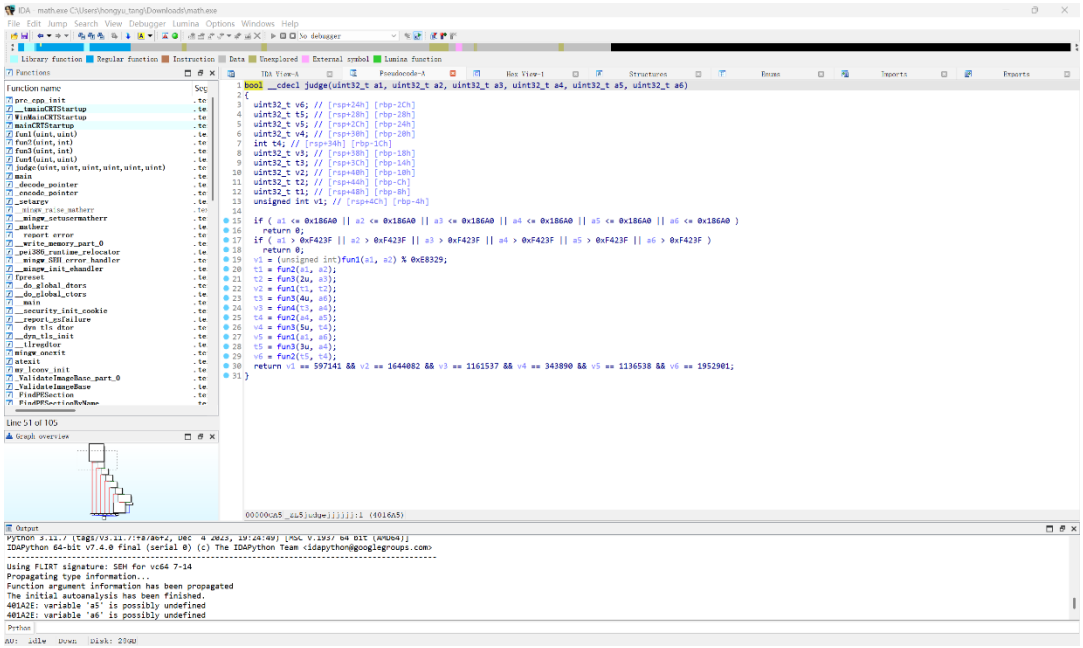
4. 得到 flag

```
[+] seed=37 => flag? BITs2CTF{pyc_reverse_is_important}
```

九、math

1. 是 64 位，拖进 ida，主要就是 judge 函数，点进去，又是四个 func 函数，解题思路应该就是解方程，懒得看了。





2. 直接 Ctrl+F5 导出所有反编译的 c 代码

3. 然后喂给 gpt 大人，gpt 大人直接输出 flag，甚至不需要我来运行脚本了。

2. (a1 - a2 + 2a3) = -190,748 + 1,834,830 = 1,644,082 （按无符号 wrap 后仍为 1,644,082）✔
3. 4a6 = 1,831,204; 1,831,204 ^ 673,893 = 1,161,537 ✔
4. 5(a4 - a5) = 5*(68,778)=343,890 ✔
5. a1 + a6 = 678,737 + 457,801 = 1,136,538 ✔
6. 2a4 + a5 = 1,347,786 + 605,115 = 1,952,901 ✔

七、Flag 格式
成功时输出： sprintf(s, "BITs2CTF{%x%x%x%x%x%x}", a, b, c, d, e, f); 注意使用的是 %x（小写十六进制，无前导 0，连在一起）。

各数十六进制（小写）： a1 = 678,737 -> 0xa5b51 a2 = 869,485 -> 0xd446d a3 = 917,415 -> 0xdffa7 a4 = 673,893 -> 0xa4865 a5 = 605,115 -> 0x93bbb a6 = 457,801 -> 0x6fc49

拼接：a5b51 d446d dffa7 a4865 93bbb 6fc49
所以最终 flag 为：
BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}

八、总结要点

- fun1 是自实现的二进制加法（体现位运算 + 进位逻辑）。
- fun2 通过加上 -a2 实现减法的 2^32 环绕，并在某些方程里故意让差为负产生环绕效果。
- fun3 是普通乘法的位分解实现。
- fun4 是 XOR。
- 题目本质是把一组位运算/模运算方程还原为代数方程，结合范围约束（六个数皆为 6 位十进制数，大于 100000 小于等于 999999），最后解出唯一一组满足的数。
- 关键点在于 (a1 - a2) 为负，从而 fun2 体现无符号环绕，这保证 v2 恰好匹配指定常量。

九、最终答案
数值： a = 678737
b = 869485
c = 917415
d = 673893
e = 605115
f = 457801

Flag： BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}

如果还需要脚本验证或想看自动求解过程（比如用 Python 约束求解），可以再继续补充。欢迎继续提问。祝解题愉快！